# 演讲稿

# 大数据用户画像与推荐分类实战 — 项目汇报演讲稿

各位老师、同学们大家好，下面由我来介绍一下我做的这个项目——**广电大数据用户画像与推荐分类实战**。

---

## 一、项目做了什么

这个项目模拟了一个广电运营商的大数据平台，从数据采集、清洗、建模、存储到可视化展示，完整跑通了整条数据链路。

数据来源是广州地区广电用户的脱敏业务数据，大约 4GB，涵盖 5 张核心业务表。最终目标是通过机器学习给每个用户打上画像标签，然后通过 Web 页面直观展示。

---

## 二、用了什么技术，核心文件和函数

整个项目按数据流分为 5 层，逐层来讲：

### 第一层：数据采集——Kafka 实时流接入

核心文件：`-trunk/user_profile_project/src/main/scala/com/tipdm/scala/chapter_3_7_streaming/KafkaStream.scala`

它的 `main` 方法里用 `KafkaUtils.createDirectStream` 直连 Kafka，采用 Direct API 无 Receiver 模式。每 10 秒一个微批次，从 `order_topic` 主题消费订单数据。

核心业务逻辑是：解析每条消息（分号分隔），取第 15 个字段作为 `cost` 金额，过滤掉 null 值和以 “YH” 开头的无效订单，用 Spark 的 **Accumulator** 累加器统计总营业额、有效订单数、新增订单数。

统计结果通过 `InternalRedisClient.getJedis()` 获取 Redis 连接，用 Pipeline + 事务（`multi/exec`）批量写入 Redis，存储键包括 `totalcost`、`totalOrders`、`increase_cost` 等。

另外，它还实现了**断点续消费**——每批次处理完后把各分区的 offset 写回 Redis，下次启动时从 Redis 读取 offset 恢复消费位置。

辅助工具文件：- `trunk/user_profile_project/src/main/scala/com/tipdm/scala/util/InternalRedisClient.scala` — Redis 连接池管理，提供 `getJedis()` 获取连接，配置从 `redis.properties` 读取

## 第二层：数据清洗——Spark ETL 离线处理

核心文件：- `trunk/user_profile_project/src/main/scala/com/tipdm/scala/chapter_3_6_processing/DataProcess.scala`

它的入口是 `main` 方法，接收 10 个参数——5 个输入表名和 5 个输出表名，然后循环调用 `dataProcessing(sqlContext, inputTable, flag, outputTable)` 方法对每张表做清洗。

清洗逻辑包括：- 全表 `distinct()` 去重 - 过滤政企用户（`owner_name` 为 EA~EE 级） - 过滤特殊线路（`owner_code` 为 02/09/10） - 只保留珠江宽频、数字电视、互动电视、甜果电视的用户

对 `media_index` 表还有专门逻辑：过滤 duration 不在 20 秒~5 小时范围内的异常收视记录，过滤编号 5401487 的异常用户。

对 `mediamatch_usermsg` 表：按 `phone_no` 取最新 `run_time` 的记录（去重取最新状态）。

清洗后的数据以 `SaveMode.Overwrite` 写入 Hive DWD 层，存储格式为 **Parquet 列式存储**。

建表脚本：- `trunk/create_ods_fixed.sql` — ODS 层建表 DDL，用 `\073` 八进制转义符替代分号声明 CSV 分隔符

ETL 执行命令：

```
        spark-submit \
          --driver-class-path /export/servers/hive-
1.2.1/lib/mysql-connector-java-5.1.45.jar \
          --class
com.tipdm.scala.chapter_3_6_processing.DataProcess \
```

```
          --master local[*] \

/export/data/big_data_case_study_project/user_profile_project-
1.0.jar \
          media_index_3m media_1d \
          mediamatch_userevent_3m mediamatch_userevent_1d \
          mediamatch_usermsg_3m mediamatch_usermsg_1d \
          mmconsume_billevents_3m mmconsume_billevents_1d \
          order_index_v3 order_index_1d
```

## 第三层：算法建模——SVM 支持向量机

涉及两个关键文件：

**特征工程：** - `trunk/user_profile_project/src/main/scala/com/tipdm/scala/processing/SVMDataProcess.scala`

它的 `main` 方法把 5 张 DWD 表通过 `phone_no` 做 LEFT JOIN 拼成宽表，然后构造三个核心特征：- `consume`：月均消费金额 = `sum(should_pay) / 3` - `join_time`：入网时长 = `months_between(当前日期, run_time) / 12`，单位年 - `count_duration`：平均每次收视时长（小时）

标签定义方面：月收视总时长超过 18720000 毫秒（约 5.2 小时/天）且有有效订单的标记为活跃用户（label=1），主动销户或暂停的标记为离网用户（label=0）。

**模型训练：** - `trunk/user_profile_project/src/main/scala/com/tipdm/scala/chapter_3_8_6_svm/SVM.scala`

它把特征数据构造成 `LabeledPoint`，用 `StandardScaler` 做归一化，然后调用 `SVMWithSGD.train()` 进行随机梯度下降训练。关键超参数：迭代次数 30，学习步长 0.1，正则化参数 0.01，miniBatchFraction 1.0。

训练完成后用 `BinaryClassificationMetrics` 计算评估指标——准确率 79.25%，AUC 0.82，PR-AUC 0.92，结果写入 Hive 的 `svm_activate` 和 `svm_prediction` 表。

执行脚本（虚拟机上）：- `/root/run_svm.sh`

```
        spark-submit \
          --driver-memory 3g \
          --conf spark.driver.maxResultSize=1g \
          --conf spark.storage.memoryFraction=0.3 \
          --driver-class-path /export/servers/hive-
1.2.1/lib/mysql-connector-java-5.1.45.jar \
          --class com.tipdm.scala.chapter_3_8_6_svm.SVM \
          --master local[*] \
```

```
/export/data/big_data_case_study_project/user_profile_project-
1.0.jar \
          mmconsume_billevents_1d mediamatch_userevent_1d media_1d
\
          mediamatch_usermsg_1d order_index_1d \
          svm_activate svm_prediction \
          30 0.1 0.01 1.0 default
```

## 第四层：结果落地——MySQL/Redis 存储

核心文件：- `trunk/user_profile_project/src/main/scala/com/tipdm/scala/chapter_3_9_3_user_profile/SQLEngine.scala` — 通用 SparkSQL 执行引擎，接收 SQL 和输出目标（hive/rdbms）- `trunk/user_profile_project/src/main/scala/com/tipdm/scala/util/SparkUtils.scala` — 工具类，提供 `saveDF2MySQL()`、`saveDF2Hive()` 等方法

执行脚本（虚拟机上）：- `/root/run_labels.sh` — 批量执行 9 个画像标签的 SQL，通过 `SQLEngine` + `DataFrame.write.jdbc()` 写入 MySQL `zjsm.user_label` 表

9 大用户画像标签：消费内容、电视消费水平、宽带消费水平、宽带产品带宽、用户是否挽留、业务品牌、电视入网程度、宽带入网程度、销售品名称。

辅助文件：- `trunk/user_profile_project/src/main/scala/com/tipdm/scala/statistics/AreaStatistics.scala` — 按广州市行政区统计用户数量

## 第五层：可视化展示——Spring Boot Web

核心文件：- `trunk/user_profile_visualization/src/main/java/com/tipdm/Application.java` — Spring Boot 启动类，端口 8001，上下文路径 `/user_profile` - `trunk/user_profile_visualization/src/main/java/com/tipdm/controller/UserLabelController.java` — REST 控制器，提供两个接口：- `GET /labels/user/{id}` → 调用 `UserLabelService.getLabelsByPhoneNo()` 返回标签列表 - `GET /labels/users/{id}/all/` → 调用 `UserLabelService.findLabel()` 返回按父级标签分组的数据 - `trunk/user_profile_visualization/src/main/java/com/tipdm/service/UserLabelService.java` — 业务层，`findLabel()` 方法从 `application.yml` 读取 9 个父级标签名，遍历用户标签按 `parent_label` 归组 - `trunk/user_profile_visualization/src/main/java/com/tipdm/repository/UserLabelRepository.java` — 数据访问层，原生 SQL `SELECT * FROM user_label WHERE phone_no=?` - `trunk/user_profile_visualization/src/main/java/com/tipdm/entity/UserLabel.java` — 实体类，`@EmbeddedId` 实现 phone_no + label + parent_label 复合主键

前端文件： - `trunk/user_profile_visualization/src/main/resources/templates/index.html` — 主页面，Vue.js 数据绑定 + Element UI 组件 - `trunk/user_profile_visualization/src/main/resources/static/css/dashboard.css` — 深色科技风大屏样式 - `trunk/user_profile_visualization/src/main/resources/templates/fragments/header.html` — 头部，引入 CSS/JS - `trunk/user_profile_visualization/src/main/resources/templates/fragments/footer.html` — 尾部，引入 Vue/ECharts/Element UI

配置文件： - `trunk/user_profile_visualization/src/main/resources/application-dev.yml` — 开发环境配置，数据库连接 `jdbc:mysql://192.168.2.162:3306/zjsm` - `trunk/user_profile_visualization/pom.xml` — Maven 依赖，Spring Boot 1.5.10 + Thymeleaf + JPA

## 支撑模块一：ELK 日志监控

虚拟机上的组件： - `/export/servers/elasticsearch-6.3.2/` — Elasticsearch，以 es 用户启动 - `/export/servers/kibana-6.3.2/` — Kibana，连接 9200 端口 - `/export/servers/filebeat-6.3.2-linux-x86_64/` — Filebeat，监听 Nginx 日志 - `/export/servers/metricbeat-6.3.2-linux-x86_64/` — Metricbeat，采集系统指标 - `/export/servers/logstash-6.3.2/` — Logstash，CSV 数据清洗

Logstash 配置文件： - `trunk/user_profile_scripts/csv2elasticsearch/media1.conf` ~ `media5.conf` — 媒体数据导入 ES - `trunk/user_profile_scripts/csv2elasticsearch/order_index.conf` — 订单数据导入 ES - `trunk/user_profile_scripts/csv2elasticsearch/mm_billevents.conf` — 账单数据导入 ES - `trunk/user_profile_scripts/csv2elasticsearch/mediamatch_usermsg.conf` — 用户信息导入 ES - `trunk/user_profile_scripts/csv2elasticsearch/mediamatch_userevent.conf` — 用户事件导入 ES

ES 数据同步文件： - `trunk/user_profile_project/src/main/scala/com/tipdm/scala/chapter_3_4_3_datasource/Hive2Elasticsearch.scala` — Hive 导出到 ES，用 `EsSparkSQL.saveToEs()` - `trunk/user_profile_project/src/main/scala/com/tipdm/scala/chatper_3_5_1_datasource/Elasticsearch2Hive.scala` — ES 导入到 Hive - `trunk/user_profile_project/src/main/scala/com/tipdm/scala/chatper_3_5_1_datasource/ElasticsearchMulti2Hive.scala` — 多个 ES 索引合并导入 Hive

### 支撑模块二：XXL-Job 定时调度

项目模块：- `trunk/timer_job_admin/` — 调度中心 Web 界面（本地 8080 端口）- `trunk/timer_job_core/` — 核心调度引擎 - `trunk/timer_job_java_executor/` — Java 执行器 - `trunk/timer_job_shell_executor/` — Shell 执行器

定时任务脚本：- `trunk/user_profile_scripts/timed_data_generation/media_index/xxl_job_shell_media_index.sh` — 媒体数据 T+1 生成，Cron: `0 10 1 * * ? *` - `trunk/user_profile_scripts/timed_data_generation/media_index/create_media_1d.hql` — 对应的 Hive SQL - `trunk/user_profile_scripts/timed_data_generation/order_index/xxl_job_shell_order_index.sh` — 订单数据 T+1 生成，Cron: `0 10 2 * * ? *` - `trunk/user_profile_scripts/timed_data_generation/order_index/create_order_index_1d.hql` — 对应的 Hive SQL - `trunk/user_profile_scripts/timed_data_generation/mmconsume_billevents/xxl_job_shell_mmconsume_billevents.sh` — 账单数据月度生成，Cron: `0 10 3 1 * ? *` - `trunk/user_profile_scripts/timed_data_generation/mmconsume_billevents/create_mmconsume_billevents_1d.hql` — 对应的 Hive SQL

ES 导出到 Kafka：- `trunk/user_profile_scripts/elasticsearch2kafka/order_kafka.conf` — 订单数据从 ES 导出到 Kafka topic `order`

### 公共依赖模块

- `trunk/spark_hadoop_common/` — Spark/Hadoop 公共依赖
- `trunk/user_profile_project/src/main/scala/com/tipdm/scala/util/SparkUtils.scala` — 工具类：`saveDF2MySQL()`、`saveDF2Hive()`、`getBeforeTimeStr()` 时间计算等
- `trunk/user_profile_project/src/main/scala/com/tipdm/scala/util/InternalRedisClient.scala` — Redis 连接池，`JedisPool` + ShutdownHook

### 虚拟机启动脚本

- `/export/servers/start_all.sh` — 一键启动所有服务（Hadoop、Kafka、Hive、ES、Kibana、Beats）

---

## 三、核心点

**第一是全链路贯通**。Kafka → Hive ODS/DWD/DWS → Spark ETL → SVM 建模 → MySQL/Redis → Spring Boot 可视化，整条数据管道闭环。

**第二是 SVM 建模**。多表 JOIN 特征工程 + StandardScaler 归一化 + SVMWithSGD 训练，AUC 达到 0.82。

**第三是实时+离线双引擎**。`KafkaStream` 做秒级实时监控，`DataProcess` + `SQLEngine` 做离线画像计算。

---

# 四、难点和遇到的问题

## 1. SVM 训练内存溢出（`GC overhead limit exceeded`）

虚拟机只有 3.7GB 内存，Spark 默认 `--driver-memory 1g` 远远不够。

**解决**：扩内存到 5.7GB，设 `--driver-memory 3g`，调 `spark.storage.memoryFraction=0.3` 减少 RDD 缓存占比。

## 2. SVM 梯度爆炸

学习步长 1.0 太大导致 SparkContext 崩溃。

**解决**：步长降到 0.1，正则化参数 0.01。

## 3. Hive 分号解析冲突

CSV 用分号分隔，但 Hive CLI 把引号内的分号也当语句终止符。

**解决**：用八进制 `\073` 替代。

## 4. Spark 找不到 Hive 表（`Table not found`）

Spark 默认用内嵌 Derby，看不到 Hive MySQL 里的元数据。

**解决**：`cp /export/servers/hive-1.2.1/conf/hive-site.xml /export/servers/spark-1.6.3/conf/` + `--driver-class-path` 指定 MySQL 驱动。

## 5. Kibana Exit 84

ES 没完全启动就启 Kibana 会直接退出。

**解决**：先 `curl localhost:9200` 确认 ES 就绪，再启动 Kibana；修改 `kibana.yml` 将 `server.host` 改为 `0.0.0.0`。

---

# 五、做完项目对自己的提升

**第一是大数据全栈实战能力**。之前对 Hadoop、Spark、Hive 这些都只停留在概念层面，通过这个项目真正动手跑通了从数据接入到可视化的全链路，对每个组件的角色和配合方式有了实战级的理解。

**第二是问题排查能力**。这个项目几乎每个模块都遇到了问题，从 JDBC 驱动缺失、类加载冲突、内存溢出到梯度爆炸，每一个都逼着我去读日志、分析堆栈、查文档。现在看到报错信息不会慌了，知道怎么一步步定位。

**第三是对机器学习工程化的理解**。算法不是调个 API 就完事的，特征工程怎么做、数据怎么清洗、模型参数怎么调、结果怎么落地服务化，这些工程层面的事情比算法本身更耗时间，但也更重要。

---

# 六、后期优化方向

1. **Spark 版本升级**：Spark 1.6 升级到 3.x，利用 AQE（Adaptive Query Execution）自适应查询优化
2. **模型升级**：SVM 换成随机森林/GBDT 提升精度
3. **流处理升级**：Spark Streaming 迁移到 Flink 实现真正事件驱动
4. **前端增强**：加入用户分群对比、实时大屏
5. **配置管理**：消除代码中硬编码的 IP 和路径，统一配置管理

---

谢谢大家！